

Reviewer Pack — Minimal Frobenius Index and SAT Clause Subset Complexity Cor...

Entry 1ddc1ab64b19 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 21:21:01 UTC

Minimal Frobenius Index and SAT Clause Subset Complexity Correlation

Entry ID: 1ddc1ab64b19 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 21:21:01 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Frobenius Index) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subset Complexity)

Statement:

For every CNF formula φ with n variables, the minimal Frobenius index of the vector space spanned by its clauses is linearly correlated with its SAT clause subset complexity, such that $\min_index(\varphi) = \Theta(\sum_c |C|^{1/k})$, where k is a constant and $|C|$ is the size of a clause C in φ .

Rationale (proposer's reasoning):

The Frobenius index measures the complexity of representations over finite fields, while SAT clause subset complexity quantifies the hardness of resolving clauses. A link between these two concepts could reveal new insights into the structure of satisfiability problems and their representations.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b0c645f8a8a7912d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the generated CNF formulas show a Spearman's rank correlation coefficient greater than or equal to 0.8 between their minimal Frobenius index and SAT clause subset complexity, with no formula having a correlation coefficient less than -0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Frobenius Index AND SAT Clause Subset Complexity - Representation Theory AND Boolean Satisfiability Correlation - CNF Formula $\min_index \ 0(\Sigma_c \ |C|^{(1/k)})$

Top relevant hits considered: - [s2:1109.2134] Generalizing Boolean Satisfiability II: Theory - [s2:10.1109/HLDVT.2003.1252475] Logic transformation and coding theory-based frameworks for Boolean satisfiability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import itertools

def generate_cnf(n, m):
    clauses = []
    for _ in range(m):
        clause = [random.randint(1, n), -random.randint(1, n)]
        clauses.append(clause)
    return clauses

def frobenius_index(cnf):
    if not cnf:
        return 0
    n = len(cnf[0])
    matrix = [[0] * (2 * n) for _ in range(n)]
    for clause in cnf:
        i, j = abs(clause[0]) - 1, abs(clause[1]) - 1
        if clause[0] > 0:
            matrix[i][2 * i] = 1
        else:
            matrix[i][2 * i + 1] = 1
        if clause[1] > 0:
            matrix[j][2 * j] = 1
        else:
            matrix[j][2 * j + 1] = 1
    for i in range(n):
        if matrix[i][i] == 0:
            return float('inf')
    return n
```

```

def sat_clause_subset_complexity(cnf):
    max_length = max(len(clause) for clause in cnf)
    return sum(2 ** (len(clause) - 1) for clause in cnf)

def spearman_correlation(x, y):
    if len(x) != len(y):
        raise ValueError("x and y must have the same length")
    n = len(x)
    sorted_x = sorted(range(n), key=lambda i: x[i])
    sorted_y = sorted(range(n), key=lambda i: y[i])
    rank_x = [sorted_x.index(i) for i in range(n)]
    rank_y = [sorted_y.index(i) for i in range(n)]
    d_squared_sum = sum((rank_x[i] - rank_y[i]) ** 2 for i in range(n))
    return 1 - (6 * d_squared_sum) / (n * (n**2 - 1))

def run_trial(seed: int):
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        cnf = generate_cnf(n, 2 * n)
        min_index = frobenius_index(cnf)
        sat_complexity = sat_clause_subset_complexity(cnf)
        if min_index == float('inf'):
            continue
        results.append((min_index, sat_complexity))
    if not results:
        return {
            "metric_name": "Spearman's Rank Correlation Coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "No valid CNF formulas generated"
        }
    x, y = zip(*results)
    correlation = spearman_correlation(x, y)
    return {
        "metric_name": "Spearman's Rank Correlation Coefficient",
        "metric_value": correlation,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": correlation >= 0.8 and all(correlation >= -0.5 for _ in results),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 100000) for _ in range(30)]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

```

```

if all(result is not None for result in results):
    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= 0.8) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result < 0.8)
        print(f"RESULT: FALSIFIED counterexample='Spearman correlation below threshold' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE some trials produced None")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_57d165d2.py", line 95, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_57d165d2.py", line 65, in run_trial
    min_index = frobenius_index(cnf)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_57d165d2.py", line 32, in frobenius_index
    matrix[i][2 * i] = 1
    ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to evaluate the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17735
2	preregistration	ollama_remote	glm4:latest	0	0	9124

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	14160
4	novelty	ollama_remote	glm4:latest	0	0	18136
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51936
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13203
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12213
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12141
9	judge	ollama_remote	glm4:latest	0	0	12423

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 161071 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/1ddc1ab64b19.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/1ddc1ab64b19.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/1ddc1ab64b19.tar.gz* (if generated)